

AI-Powered Financial Intelligence Platform

Overview

This project presents an end-to-end Financial Business Intelligence solution designed to simulate how modern banking and financial institutions use Artificial Intelligence for decision-making.

The platform combines machine learning, forecasting, anomaly detection, explainable AI, and business intelligence reporting into one integrated system.

The objective was to build a solution that not only predicts business performance but also explains outcomes in a transparent way suitable for executive decision-making.

Business Problem

Financial organisations process millions of transactions every day and need intelligent systems to answer questions such as:

How much revenue can be expected in the coming months? Which transactions appear unusual? What factors influence business performance? How can AI decisions be explained to stakeholders? This project addresses these challenges through a complete analytics pipeline.

Project Objectives

Forecast future revenue trends Detect anomalous transactions Explain AI-generated predictions Present insights through an executive dashboard System Architecture

Synthetic Financial Data ↓ SQL Storage ↓ Data Processing ↓ Forecasting Model ↓ Fraud Detection Model ↓ Explainable AI ↓ Power BI Dashboard

Technology Stack

Python PostgreSQL Power BI Prophet Isolation Forest SHAP Pandas Scikit-Learn

Features Revenue Forecasting Predicts future financial performance using time-series forecasting.

Fraud Detection Identifies unusual transaction behaviour.

Explainable AI Provides interpretation behind model predictions.

Executive Dashboard Displays KPIs and business metrics.

Results The project successfully demonstrates how machine learning and business intelligence can be integrated into a practical financial analytics environment.

Future Improvements Real-time prediction APIs Cloud deployment Streaming data ingestion Deep learning forecasting.

Yaswanth Chinthala

```
import pandas as pd
import numpy as np

np.random.seed(42)
rows = 50000

data = pd.DataFrame()

data["transaction_id"] = np.arange(1, rows + 1)

data["date"] = pd.date_range(
    start="2024-01-01",
    periods=rows,
    freq="h"
)

data["customer_id"] = np.random.randint(10000, 99999, rows)

data["transaction_amount"] = np.random.normal(5000, 1500, rows)

data["account_balance"] = np.random.normal(80000, 25000, rows)

data["region"] = np.random.choice(["North", "South", "East", "West"], rows)

data["product"] = np.random.choice(["Investments", "Insurance", "Loans", "Savings"], rows)

data["channel"] = np.random.choice(["ATM", "Branch", "Online", "Mobile"], rows)

data["revenue"] = data["transaction_amount"] * np.random.uniform(0.02, 0.08, rows)

data["fraud_label"] = np.random.choice([0, 1], rows, p=[0.97, 0.03])

data["kpi_score"] = np.random.uniform(10, 100, rows).round(2)
```

```
# Save directly to Colab's working directory (no subfolder needed)
data.to_csv("bank_transactions.csv", index=False)

print("Synthetic banking dataset generated")
print(data.shape)
data.head()
```

Synthetic banking dataset generated
(50000, 11)

	transaction_id	date	customer_id	transaction_amount	account_balance	region	product	channel	revenue	fraud_
0	1	2024-01-01 00:00:00	25795	5018.296645	118386.479434	West	Investments	Branch	352.374183	
1	2	2024-01-01 01:00:00	10860	2750.655174	44125.944100	North	Savings	Mobile	153.661189	
2	3	2024-01-01 02:00:00	86820	4003.746627	110993.943240	North	Savings	ATM	148.109915	
3	4	2024-01-01 03:00:00	64886	2148.734064	123823.819654	East	Savings	Online	44.692440	
4	5	2024-01-01 04:00:00	16265	4822.340188	118058.690930	North	Loans	Mobile	357.653110	

Next steps: [Generate code with data](#) [New interactive sheet](#)

```
import pandas as pd

# Load from Colab's working directory (no 'data/' subfolder)
df = pd.read_csv("bank_transactions.csv")

print("Shape before cleaning:", df.shape)
print("\nColumn names:", df.columns.tolist())
print("\nMissing values:\n", df.isnull().sum())
print("\nDuplicate rows:", df.duplicated().sum())

# Drop duplicates
df.drop_duplicates(inplace=True)

# Drop nulls
df.dropna(inplace=True)

# Parse date column
df["date"] = pd.to_datetime(df["date"])

# Extract useful time features
df["year"] = df["date"].dt.year
df["month"] = df["date"].dt.month
df["day"] = df["date"].dt.day
df["hour"] = df["date"].dt.hour

# Ensure correct data types
df["transaction_id"] = df["transaction_id"].astype(int)
df["customer_id"] = df["customer_id"].astype(int)
df["fraud_label"] = df["fraud_label"].astype(int)
df["transaction_amount"] = df["transaction_amount"].astype(float)
df["account_balance"] = df["account_balance"].astype(float)
df["revenue"] = df["revenue"].astype(float)
df["kpi_score"] = df["kpi_score"].astype(float)

# Remove any rows with negative transaction amounts or balances (sanity check)
df = df[df["transaction_amount"] > 0]
df = df[df["account_balance"] > 0]

# Save cleaned file to working directory
df.to_csv("cleaned_data.csv", index=False)

print("\nPreprocessing complete")
print("Shape after cleaning:", df.shape)
df.head()
```

Shape before cleaning: (50000, 11)

Column names: ['transaction_id', 'date', 'customer_id', 'transaction_amount', 'account_balance', 'region', 'product', 'channel', 'revenue', 'fraud_label', 'kpi_score']

Missing values:

```
transaction_id    0
date              0
customer_id       0
transaction_amount 0
account_balance   0
region            0
product           0
channel           0
revenue           0
fraud_label       0
kpi_score         0
dtype: int64
```

Duplicate rows: 0

Preprocessing complete

Shape after cleaning: (49943, 15)

	transaction_id	date	customer_id	transaction_amount	account_balance	region	product	channel	revenue	fraud_label
0	1	2024-01-01 00:00:00	25795	5018.296645	118386.479434	West	Investments	Branch	352.374183	
1	2	2024-01-01 01:00:00	10860	2750.655174	44125.944100	North	Savings	Mobile	153.661189	
2	3	2024-01-01 02:00:00	86820	4003.746627	110993.943240	North	Savings	ATM	148.109915	
3	4	2024-01-01 03:00:00	64886	2148.734064	123823.819654	East	Savings	Online	44.692440	
4	5	2024-01-01 04:00:00	16265	4822.340188	118058.690930	North	Loans	Mobile	357.653110	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
import sqlite3
import pandas as pd
import os

os.makedirs("outputs", exist_ok=True)

# — Connect to SQLite database (creates file if not exists) —————
conn = sqlite3.connect("outputs/financial_data.db")
cursor = conn.cursor()

# — Drop table if re-running to avoid conflicts —————
cursor.execute("DROP TABLE IF EXISTS financial_data;")

# — Create table (SQLite syntax) —————
cursor.execute("""
CREATE TABLE financial_data (
    transaction_id    INTEGER PRIMARY KEY AUTOINCREMENT,
    date              TEXT,
    customer_id       INTEGER,
    transaction_amount REAL,
    account_balance   REAL,
    region            TEXT,
    product           TEXT,
    channel           TEXT,
    revenue           REAL,
    fraud_label       INTEGER,
    kpi_score         REAL
);
""")

# — Create indexes —————
cursor.execute("CREATE INDEX IF NOT EXISTS idx_date    ON financial_data(date);")
cursor.execute("CREATE INDEX IF NOT EXISTS idx_fraud   ON financial_data(fraud_label);")
cursor.execute("CREATE INDEX IF NOT EXISTS idx_customer ON financial_data(customer_id);")

conn.commit()
print("✅ Table and indexes created successfully")
```

```

# — Load cleaned CSV and insert into SQLite —————
df = pd.read_csv("cleaned_data.csv")

# Drop transaction_id from CSV if it exists (SQLite will auto-generate it)
if "transaction_id" in df.columns:
    df = df.drop(columns=["transaction_id"])

# Keep only the columns that match the table schema
schema_columns = [
    "date", "customer_id", "transaction_amount",
    "account_balance", "region", "product",
    "channel", "revenue", "fraud_label", "kpi_score"
]
df = df[schema_columns]

# Insert all rows
df.to_sql("financial_data", conn, if_exists="append", index=False)
print(f"✅ {len(df):,} rows inserted into financial_data table")

# — Run sample queries to verify —————

print("\n— Total Rows —————")
result = pd.read_sql("SELECT COUNT(*) AS total_rows FROM financial_data;", conn)
print(result.to_string(index=False))

print("\n— Revenue by Region —————")
result = pd.read_sql("""
SELECT
    region,
    ROUND(SUM(revenue), 2) AS total_revenue,
    ROUND(AVG(transaction_amount), 2) AS avg_transaction
FROM financial_data
GROUP BY region
ORDER BY total_revenue DESC;
""", conn)
print(result.to_string(index=False))

print("\n— Fraud Summary —————")
result = pd.read_sql("""
SELECT
    fraud_label,
    COUNT(*) AS total_cases,
    ROUND(AVG(transaction_amount), 2) AS avg_amount
FROM financial_data
GROUP BY fraud_label;
""", conn)
print(result.to_string(index=False))

print("\n— Top 5 Products by Revenue —————")
result = pd.read_sql("""
SELECT
    product,
    ROUND(SUM(revenue), 2) AS total_revenue,
    COUNT(*) AS transaction_count
FROM financial_data
GROUP BY product
ORDER BY total_revenue DESC
LIMIT 5;
""", conn)
print(result.to_string(index=False))

print("\n— Channel Performance —————")
result = pd.read_sql("""
SELECT
    channel,
    COUNT(*) AS total_transactions,
    ROUND(SUM(revenue), 2) AS total_revenue,
    ROUND(AVG(kpi_score), 2) AS avg_kpi_score
FROM financial_data
GROUP BY channel
ORDER BY total_revenue DESC;
""", conn)
print(result.to_string(index=False))

# — Close connection —————
conn.close()
print("\n✅ Database saved to outputs/financial_data.db")

```

- ✅ Table and indexes created successfully
- ✅ 49,943 rows inserted into financial_data table

— Total Rows —————

```

total_rows
49943

--- Revenue by Region ---
region  total_revenue  avg_transaction
North    3127930.85      4994.51
East     3127298.03      4990.02
South    3120109.22      4988.97
West     3117071.23      5012.69

--- Fraud Summary ---
fraud_label  total_cases  avg_amount
0            48418      4995.50
1            1525      5028.91

--- Top 5 Products by Revenue ---
product  total_revenue  transaction_count
Loans     3152650.72      12577
Savings   3140780.74      12585
Investments 3116336.41      12490
Insurance  3082641.44      12291

--- Channel Performance ---
channel  total_transactions  total_revenue  avg_kpi_score
ATM              12512      3152862.29      54.92
Online           12528      3135346.11      54.88
Mobile           12464      3116399.98      55.04
Branch           12439      3087800.94      54.69

```

✓ Database saved to outputs/financial_data.db

```

import pandas as pd
from prophet import Prophet
import os

os.makedirs("outputs", exist_ok=True)

df = pd.read_csv("cleaned_data.csv")

# Parse date and aggregate revenue by date only (no time)
df["date"] = pd.to_datetime(df["date"])
df["date_only"] = df["date"].dt.date

forecast = (
    df
    .groupby("date_only")["revenue"]
    .sum()
    .reset_index()
)

forecast.columns = ["ds", "y"]
forecast["ds"] = pd.to_datetime(forecast["ds"])

# Remove any negative revenue rows
forecast = forecast[forecast["y"] > 0]

model = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=True,
    daily_seasonality=False
)

model.fit(forecast)

future = model.make_future_dataframe(periods=180)

prediction = model.predict(future)

# Save only key columns
prediction[["ds", "yhat", "yhat_lower", "yhat_upper"]].to_csv(
    "outputs/revenue_forecast.csv", index=False
)

print("Forecast completed")
print(prediction[["ds", "yhat", "yhat_lower", "yhat_upper"]].tail())

```

```

Forecast completed
      ds      yhat  yhat_lower  yhat_upper
2259 2030-03-09 5951.314335 5231.246954 6644.644395
2260 2030-03-10 6021.305564 5247.860551 6716.137394
2261 2030-03-11 5997.256836 5280.281478 6730.352142
2262 2030-03-12 5961.483152 5229.459202 6660.429739
2263 2030-03-13 5936.474248 5153.149541 6673.741637

```

```

import pandas as pd
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
import os

os.makedirs("outputs", exist_ok=True)

df = pd.read_csv("cleaned_data.csv")

# Use more features for better fraud detection
features = ["transaction_amount", "account_balance", "revenue", "kpi_score"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[features])

model = IsolationForest(
    contamination=0.03, # matches fraud_label ~3% rate in dataset
    random_state=42,
    n_estimators=100
)

df["fraud_prediction"] = model.fit_predict(X_scaled)

# Convert: IsolationForest returns -1 (fraud) and 1 (normal) → map to 1/0
df["fraud_prediction"] = df["fraud_prediction"].map({1: 0, -1: 1})

# Compare with actual fraud labels
print("Fraud prediction distribution:\n", df["fraud_prediction"].value_counts())
print("Actual fraud label distribution:\n", df["fraud_label"].value_counts())

df.to_csv("outputs/fraud_results.csv", index=False)

print("Fraud analysis completed")

```

```

Fraud prediction distribution:
fraud_prediction
0    48444
1     1499
Name: count, dtype: int64
Actual fraud label distribution:
fraud_label
0    48418
1     1525
Name: count, dtype: int64
Fraud analysis completed

```

```

import pandas as pd
import shap
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import numpy as np
import os

os.makedirs("outputs", exist_ok=True)

# — Load data —————
df = pd.read_csv("cleaned_data.csv")

features = ["transaction_amount", "account_balance", "revenue", "kpi_score"]
target = "fraud_label"

X = df[features]
y = df[target]

# — Train / test split —————
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y # stratify keeps fraud ratio balanced
)

# — Train model —————
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight="balanced" # handles imbalanced fraud_label (97% / 3%)
)
model.fit(X_train, y_train)

# — Evaluation —————
y_pred = model.predict(X_test)
print("Classification Report:\n")
print(classification_report(y_test, y_pred, target_names=["Normal", "Fraud"]))

```

```

# — SHAP on a sample for speed —————
sample = X_test.sample(500, random_state=42)

explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(sample) # returns list [class0_array, class1_array]

# — Handle both old and new SHAP API —————
# New SHAP (>0.41): shap_values is a 3D array (samples, features, classes)
# Old SHAP (< 0.41): shap_values is a list [class0, class1]
if isinstance(shap_values, list):
    sv_fraud = shap_values[1] # old API → index by class
else:
    sv_fraud = shap_values[:, :, 1] # new API → slice last dimension

# — Plot 1: Summary bar plot (mean |SHAP|) —————
shap.summary_plot(sv_fraud, sample, plot_type="bar", show=False)
plt.title("Feature Importance (SHAP) – Fraud Class")
plt.tight_layout()
plt.savefig("outputs/shap_bar.png", dpi=150, bbox_inches="tight")
plt.close()
print("Saved → outputs/shap_bar.png")

# — Plot 2: Beeswarm / dot summary plot —————
shap.summary_plot(sv_fraud, sample, show=False)
plt.title("SHAP Summary – Fraud Class")
plt.tight_layout()
plt.savefig("outputs/shap_summary.png", dpi=150, bbox_inches="tight")
plt.close()
print("Saved → outputs/shap_summary.png")

# — Plot 3: Waterfall for single prediction —————
# Works on both old and new SHAP
try:
    explanation = shap.Explanation(
        values = sv_fraud[0],
        base_values = explainer.expected_value[1],
        data = sample.iloc[0].values,
        feature_names = features
    )
    shap.plots.waterfall(explanation, show=False)
    plt.tight_layout()
    plt.savefig("outputs/shap_waterfall.png", dpi=150, bbox_inches="tight")
    plt.close()
    print("Saved → outputs/shap_waterfall.png")
except Exception as e:
    print(f"Waterfall plot skipped: {e}")

print("\nSHAP explainability completed")

```

Classification Report:

	precision	recall	f1-score	support
Normal	0.97	1.00	0.98	9684
Fraud	0.00	0.00	0.00	305
accuracy			0.97	9989
macro avg	0.48	0.50	0.49	9989
weighted avg	0.94	0.97	0.95	9989

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/tmp/ipykernel_3437/105888704.py:54: FutureWarning: The NumPy global RNG was seeded by calling `np.random.seed`. In a future
shap.summary_plot(sv_fraud, sample, plot_type="bar", show=False)
Saved → outputs/shap_bar.png
/tmp/ipykernel_3437/105888704.py:62: FutureWarning: The NumPy global RNG was seeded by calling `np.random.seed`. In a future
shap.summary_plot(sv_fraud, sample, show=False)
Saved → outputs/shap_summary.png
Saved → outputs/shap_waterfall.png

```

SHAP explainability completed

